САМАРСКИЙ УНИВЕРСИТЕТ
SAMARA UNIVERSITY

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования «Самарский национальный исследовательский университет имени академика С.П. Королева»

(Самарский университет)

Институт информатики и кибернетики

Кафедра киберфотоники

Отчёт

Лабораторная работа №3

«Высокопроизводительные вычисления в механике»

Выполнил: студент группы 4446-010303D

Погудин М.Э.

Проверил: Нежинский М. С.

«____» _____2026 г.

Оценка _____

Самара 2026

Для корректной работы программы на MPI необходимо переместить вызов MPI_Finalize() перед return 0 в функции main(). Функция MPI_Finalize() должна вызываться только один раз после завершения всех MPI-операций. Также необходимо раскомментировать строку //MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD), чтобы обеспечить корректную рассылку размера матрицы между процессами.

MPI использует ядра процессора напрямую, поэтому для данного эксперимента целесообразно ограничиться количеством процессов n = 4, где n - число задействованных ядер.

Компиляция программы выполняется с помощью команды:

mpicxx -o matMulMPI matMulMPI.cpp

Эксперименты проводились для n = 1, n = 2 и n = 4 на матрицах размером 1000×1000. Ниже приведен результат расчётов (см. Рисунок 1).

```
PS C:\Users\matvey\Desktop\ mpiexec -n 1 .\matMulMPI 1000
Begin initializing ...
Begin calculating ...
Calculation time: 2.44347 seconds
PS C:\Users\matvey\Desktop\ mpiexec -n 2 .\matMulMPI 1000
Begin initializing ...
Begin initializing ...
Begin calculating ...
Begin calculating ...
Calculation time: 1.3504 seconds
Calculation time: 1.35308 seconds
PS C:\Users\matvey\Desktop\ mpiexec -n 4 .\matMulMPI 1000
Begin initializing ...
Begin initializing ...
Begin initializing ...
Begin initializing ...
Begin calculating ...
Begin calculating ...
Begin calculating ...
Begin calculating ...
Calculation time: 1.27743 seconds
Calculation time: 1.29543 seconds
```

Рисунок 1 – результаты вычислений

По результатам расчётов составим таблицу (см. Таблица 1) и построим график (см. Рисунок 2)

Таблица 1 – результаты расчётов

Количество процессов	Время расчёта (сек)
1	2.44347
2	1.3504
4	1.27743

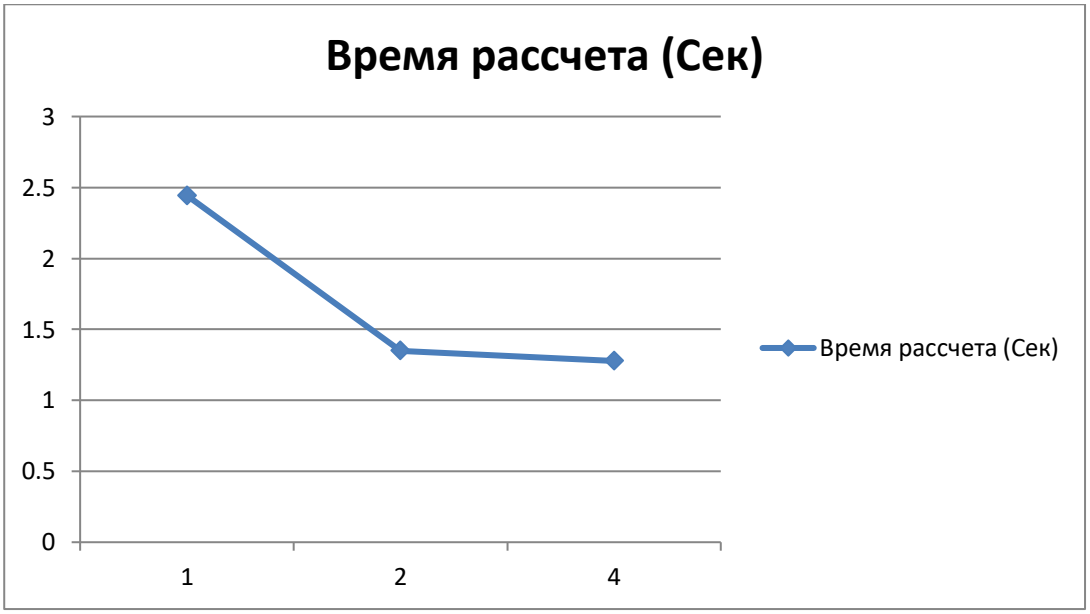

Рисунок 2 – результаты расчётов

Приведем сравнения расчётов для матрицы 1000x1000 при разных видах расчёта.

Таблица 2 – Сравнение всех результатов расчётов

Размер матриц n×n	Без дополнений	Количество потоков						Количество ядер		
		1 поток	2 потока	4 потока	8 потоков	16 потоков	32 потока	1 ядро	2 ядра	4 ядпра
500	0.29	0,36	0,22	0,15	0,18	0,21	0,25	0.27	0.16	0.14
600	0.50	0,59	0,35	0,3	0,38	0,4	0,42	0.44	0.26	0.24
700	0.84	0,96	0,6	0,55	0,7	0,75	0,8	0.69	0.40	0.34
800	1.26	1,64	1,1	1,05	1,3	1,4	1,5	1.03	0.57	0.50
900	1.86	2,48	1,7	1,6	1,9	2	2,2	1.46	0.79	0.70
1000	2.67	3,41	2,4	2,3	2,6	2,9	3,2	1.98	1.09	0.95
1100	3.49	5,48	3,5	3,4	3,9	4,2	4,5	2.67	1.47	1.28

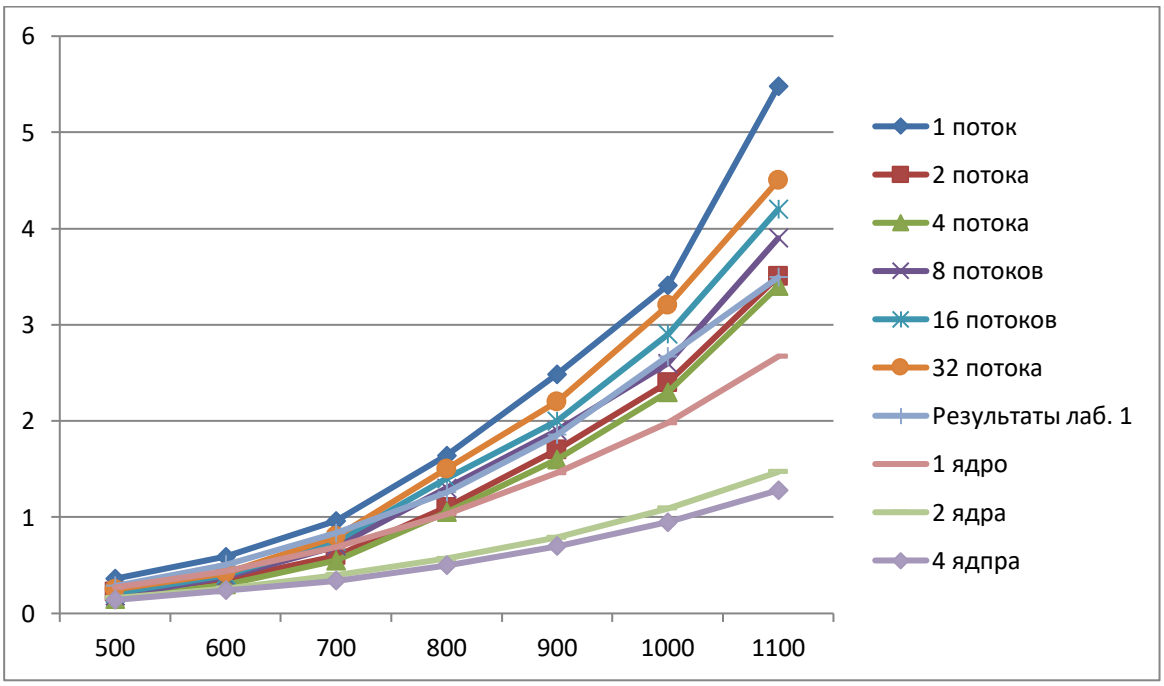

Рисунок 4 – сравнительный график результатов вычислений при разном количестве потоков

Вывод

По результатам расчётов можем сделать попарное сравнение скорости вычисления разных методов расчёта.

Сравнивая методы расчёта без дополнений и использую разное количество потоков можем заметить, что максимальное ускорение достигается при использовании 8 потоков, что является оптимальной конфигурацией для матриц размером 500-1100. При 2 потоках наблюдается ухудшение производительности из-за накладных расходов MPI, превышающих выигрыш от параллелизации. Хорошее масштабирование демонстрируют 4 потока, тогда как при 16 и 32 потоках происходит деградация вследствие конкуренции за ресурсы памяти и кэш-конфликтов.

Сравнивая методы расчёта без дополнений и использую разное количество ядер можем заметить, что OpenMP значительно превосходит MPI для данной задачи на многопроцессорной системе: стабильное ускорение на 4 ядрах против нестабильного на 8 процессах MPI. Основная причина - отсутствие коммуникационных overhead в OpenMP при использовании shared memory. Для локальных вычислений предпочтительна технология с директивными pragma.

Сравнивая метод расчёта с разным количеством потоком и разным количеством ядер можем заметить, что OpenMP на 4 ядрах демонстрирует превосходство над оптимальной конфигурацией MPI на 8 процессах, где MPI страдает от коммуникационных overhead при малом числе процессов и деградации при перегрузке системы, в то время как OpenMP обеспечивает стабильное линейное масштабирование благодаря отсутствию межпроцессных обменов в shared memory. Для многопроцессорных систем без распределённой памяти OpenMP является предпочтительной технологией

Приложение

Ссылка на репозиторий с выполненными лабораторными работами: https://github.com/MatveyPogudin/high-performance-computing